YNOV CAMPUS / JOUR 2

Jour 2 Poser des questions précises

Jointures, agrégation, sous-requêtes et opérations ensemblistes. Apprenez avec des exemples commentés, puis exercez-vous dans le TP02.

PostgreSQL 18 | 2e année de bachelor | Édition pédagogique progressive

Une jointure relie une commande à son étudiant

Une commande stocke un identifiant d’étudiant, tandis que le nom de l’étudiant appartient à students. Une jointure combine les lignes de ces tables lorsque leurs identifiants correspondent. La clé de jointure est la colonne qui les relie. Avant d’écrire le SQL, indiquez la granularité : ce que représente une ligne de résultat. Ici, une ligne représente une commande ; Alice peut donc légitimement apparaître deux fois.

Étudiants : sélection de lignes du jeu de données de référence

student_id	full_name
1	Alice Martin
2	Bilal Amrani

Choisissez le sens d’une ligne de sortie avant de choisir une jointure.

Une jointure relie une commande à son étudiant / résultat commenté

Une commande stocke un identifiant d’étudiant, tandis que le nom de l’étudiant appartient à students. Une jointure combine les lignes de ces tables lorsque leurs identifiants correspondent. La clé de jointure est la colonne qui les relie. Avant d’écrire le SQL, indiquez la granularité : ce que représente une ligne de résultat. Ici, une ligne représente une commande ; Alice peut donc légitimement apparaître deux fois.

Commandes : sélection de lignes du jeu de données de référence

order_id	student_id
101	1
102	2
103	1

Choisissez le sens d’une ligne de sortie avant de choisir une jointure.

Une jointure relie une commande à son étudiant / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

Jointure — Moyen de combiner des lignes correspondantes de plusieurs tables.

Clé de jointure — Colonne ou groupe de colonnes utilisé pour faire correspondre les lignes.

Granularité — Ce que représente une ligne d’un résultat.

Cardinalité — Nombre de lignes d’une table ou d’un résultat.

Choisissez le sens d’une ligne de sortie avant de choisir une jointure.

INNER JOIN conserve les paires qui satisfont ON

INNER JOIN renvoie une ligne pour chaque paire correspondante. ON introduit la condition de correspondance ; le signe égal exige le même identifiant d’étudiant des deux côtés. AS donne à une table un alias temporaire, ou nom court, dans cette requête. Le point dans s.full_name identifie la table de la colonne. WHERE limite ensuite ce premier exemple aux commandes 101, 102 et 103.

SQL / lire l’instruction une clause à la fois

```sql
SELECT o.order_id, s.full_name
FROM campus.orders AS o
INNER JOIN campus.students AS s
  ON s.student_id = o.student_id
WHERE o.order_id IN (101, 102, 103)
ORDER BY o.order_id;
```

Sens des mots-clés

INNER JOIN — Conserver chaque paire satisfaisant la condition de correspondance.

ON — Introduire la condition faisant correspondre deux lignes.

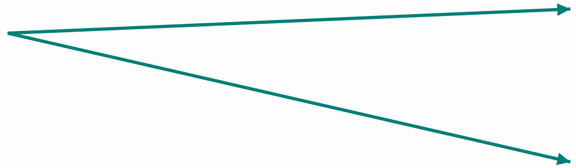
Alias de table — Nom temporaire de table utilisé dans une requête.

s.full_name — Colonne full_name de la table ayant l’alias s.

Voir comment une jointure répète un étudiant correspondant

Une jointure ne colle pas deux tables entières ligne par ligne. Elle forme les paires satisfaisant la condition ON. Alice a l’identifiant 1 et deux commandes affichées contiennent student_id 1. Elle apparaît donc deux fois dans un résultat par commande. Cette répétition est correcte au sens demandé pour les lignes ; elle ne signifie pas forcément que des étudiants sont stockés en double.

student_id	student
1	Alice
2	Bilal



order_id	student_id
101	1
102	2
103	1

L’identifiant 1 correspond à deux lignes de commande : Alice apparaît une fois par commande correspondante.

Le résultat contient une ligne par commande correspondante

Les commandes 101 et 103 appartiennent toutes deux à Alice ; son nom apparaît donc deux fois. Ce n’est pas un doublon accidentel : les identifiants de commande diffèrent et notre granularité choisie est une commande. Les trois commandes sélectionnées ont toutes un étudiant correspondant. Si une commande n’en avait pas, cette jointure INNER JOIN l’omettrait au lieu d’inventer un nom.

Résultat / données d’exemple

order_id	full_name
101	Alice Martin
102	Bilal Amrani
103	Alice Martin

Les noms répétés peuvent être corrects lorsque la sortie représente des commandes.

USING nomme une clé d’égalité commune

Lorsque les deux tables utilisent le même nom de clé de jointure, USING demande plus brièvement l’égalité sur cette colonne. JOIN seul signifie INNER JOIN. USING (department_id) fait correspondre le department_id de chaque étudiant au department_id du département. Il expose un seul department_id commun au lieu de deux copies. ON reste utile si les noms diffèrent ou si la condition nécessite plus que ces égalités nommées.

SQL / lire l’instruction une clause à la fois

```
SELECT s.full_name, d.name AS department
FROM campus.students AS s
JOIN campus.departments AS d
  USING (department_id)
WHERE s.student_id <= 3
ORDER BY s.student_id;
```

Résultat / données d’exemple

full_name	department
Alice Martin	Computing
Bilal Amrani	Computing
Chloe Bernard	Design

USING nomme une clé d’égalité commune / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

USING (column) — Faire correspondre les valeurs égales de la colonne de même nom des deux côtés.

Colonne de sortie commune — Clé de jointure affichée une seule fois par USING, au lieu de deux copies.

LEFT JOIN préserve chaque ligne à sa gauche

Une jointure LEFT JOIN conserve chaque ligne de la table écrite à sa gauche. Les lignes correspondantes de droite sont rattachées comme d’habitude. Sans correspondance, SQL fournit NULL pour les colonnes de droite. NULL signifie manquant ou inconnu, pas zéro. Ici, students est à gauche, car la liste de contacts doit inclure les étudiants même s’ils n’ont jamais passé commande.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.full_name, o.order_id, o.status
FROM campus.students AS s
LEFT JOIN campus.orders AS o
  ON o.student_id = s.student_id
WHERE s.student_id >= 5
ORDER BY s.student_id, o.order_id;
```

Sens des mots-clés

LEFT JOIN — Garder les lignes de gauche ; compléter par NULL à droite sans correspondance.

Ligne sans correspondance — Ligne pour laquelle la condition de jointure ne trouve aucun partenaire.

Farah reste dans la jointure sans commande

Eva apparaît deux fois, car les commandes 106 et 109 correspondent toutes deux à son identifiant d’étudiant. Farah apparaît une fois avec NULL dans les colonnes de commande, car aucune commande ne correspond. Le mot NULL affiché représente une valeur nulle SQL dans ces tables pédagogiques. Ce n’est pas le texte stocké dans une colonne. LEFT JOIN préserve les lignes de gauche, mais plusieurs correspondances à droite peuvent augmenter le nombre de lignes.

Résultat / données d’exemple

full_name	order_id	status
Eva Dupont	106	cancelled
Eva Dupont	109	paid
Farah Idrissi	NULL	NULL

Préserver une ligne de gauche n’implique pas une ligne de sortie par ligne de gauche.

Placer une condition de correspondance facultative dans ON

Nous voulons maintenant chaque étudiant sélectionné, avec uniquement ses commandes payées rattachées. Placer o.status = 'paid' dans ON limite les commandes pouvant correspondre tout en préservant Farah. Déplacer cette condition dans WHERE supprimerait Farah : le statut NULL fourni pour elle ne rend pas la condition vraie. ON décide quelles paires correspondent ; WHERE décide quelles lignes du résultat terminé restent.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.full_name, o.order_id
FROM campus.students AS s
LEFT JOIN campus.orders AS o
  ON o.student_id = s.student_id
 AND o.status = 'paid'
WHERE s.student_id >= 5
ORDER BY s.student_id, o.order_id;
```

Résultat / données d’exemple

full_name	order_id
Eva Dupont	109
Farah Idrissi	NULL

Sens des mots-clés

Prédicat — Condition utilisée pour tester des lignes ou des paires de lignes.

Différence entre ON et WHERE — ON contrôle la correspondance ; WHERE filtre les lignes après la jointure.

À vous de jouer : trouver les étudiants sans commande

L’équipe de communication veut les étudiants qui n’ont jamais passé de commande, quel que soit son statut. Partez de students à gauche et orders à droite. Utilisez LEFT JOIN sur student_id, puis testez order_id à droite avec IS NULL. Renvoyez student_id et full_name, triés par student_id. Prévoyez le nom avant d’exécuter la requête et expliquez pourquoi tester l’identifiant de l’étudiant ne fonctionnerait pas.

Suivez le raisonnement

1. Choisissez students comme table de gauche préservée.
2. Faites correspondre les commandes avec ON o.student_id = s.student_id.
3. Conservez uniquement les lignes où, à droite, o.order_id IS NULL.

Utilisez une clé non nulle du côté facultatif pour détecter une ligne sans correspondance.

Corrigé expliqué : Farah n’a aucune commande correspondante

Farah est la seule étudiante sans commande dans le jeu de données de référence. La jointure crée d’abord pour elle une ligne préservée, puis IS NULL la conserve. Les autres étudiants ont au moins une commande : leurs valeurs order_id sont présentes et WHERE élimine leurs lignes. Tester s.student_id IS NULL ne trouverait personne, car l’identifiant de l’étudiant préservé reste présent.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.student_id, s.full_name
FROM campus.students AS s
LEFT JOIN campus.orders AS o
  ON o.student_id = s.student_id
WHERE o.order_id IS NULL
ORDER BY s.student_id;
```

Résultat / données d’exemple

student_id	full_name
6	Farah Idrissi

RIGHT et FULL choisissent les côtés préservés

RIGHT JOIN préserve la table de droite ; inverser les tables et utiliser LEFT JOIN exprime le même choix. FULL JOIN préserve les lignes sans correspondance des deux côtés. OUTER est facultatif dans les deux écritures. Ces petites listes VALUES représentent une liste de membres et des inscriptions. Les parenthèses permettent à chaque résultat VALUES d’agir comme table FROM. AS a(name) nomme cette source de lignes a et sa colonne name ; b(name) nomme l’autre liste de même.

SQL / lire l’instruction une clause à la fois

```sql
SELECT a.name AS roster, b.name AS signup
FROM (VALUES ('Alice'), ('Bilal')) AS a(name)
FULL JOIN (VALUES ('Bilal'), ('Chloe')) AS b(name)
  ON a.name = b.name
ORDER BY a.name, b.name;
SELECT a.name AS roster, b.name AS signup
FROM (VALUES ('Alice'), ('Bilal')) AS a(name)
RIGHT JOIN (VALUES ('Bilal'), ('Chloe')) AS b(name)
  ON a.name = b.name
ORDER BY a.name, b.name;
```

Sens des mots-clés

RIGHT JOIN — Conserver chaque ligne de droite, avec ou sans correspondance.

FULL JOIN — Conserver les correspondances et les lignes sans correspondance des deux côtés.

OUTER — Mot facultatif dans LEFT, RIGHT et FULL JOIN.

(VALUES ...) AS a(name) — Utiliser des lignes littérales comme table a avec une colonne nommée name.

FULL révèle les deux types de correspondance manquante

Bilal correspond, car son nom apparaît dans les deux listes. Alice est uniquement dans la liste de membres ; FULL JOIN fournit donc une inscription NULL. Chloe est uniquement inscrite ; FULL JOIN fournit donc un membre NULL. RIGHT JOIN conserve Bilal et Chloe, tous deux dans la liste d’inscriptions de droite, mais élimine Alice. Choisissez la jointure en décidant quelles correspondances manquantes la question métier doit préserver.

Résultat de FULL JOIN

roster	signup
Alice	NULL
Bilal	Bilal
NULL	Chloe

FULL révèle les deux types de correspondance manquante / résultat commenté

Bilal correspond, car son nom apparaît dans les deux listes. Alice est uniquement dans la liste de membres ; FULL JOIN fournit donc une inscription NULL. Chloe est uniquement inscrite ; FULL JOIN fournit donc un membre NULL. RIGHT JOIN conserve Bilal et Chloe, tous deux dans la liste d’inscriptions de droite, mais élimine Alice. Choisissez la jointure en décidant quelles correspondances manquantes la question métier doit préserver.

Résultat de RIGHT JOIN

roster	signup
Bilal	Bilal
NULL	Chloe

CROSS JOIN génère délibérément toutes les paires

CROSS JOIN associe chaque ligne de gauche à chaque ligne de droite ; il n’a pas de condition ON. C’est utile lorsque toutes les combinaisons sont voulues, comme pour prévoir deux produits pour deux campus. Deux lignes de gauche multipliées par deux de droite produisent quatre lignes de résultat. L’oubli accidentel d’une condition de correspondance peut créer la même multiplication alors que la question ne la demandait pas.

SQL / lire l’instruction une clause à la fois

```sql
SELECT p.name, c.campus
FROM campus.products AS p
CROSS JOIN (VALUES ('north'), ('south'))
  AS c(campus)
WHERE p.product_id IN (1, 2)
ORDER BY p.product_id, c.campus;
```

Sens des mots-clés

CROSS JOIN — Renvoyer toutes les paires possibles de lignes de gauche et de droite.

Produit cartésien — Autre nom de toutes les combinaisons possibles de lignes.

Deux produits multipliés par deux campus donnent quatre lignes

Chaque produit apparaît une fois pour north et une fois pour south. Aucune relation de campus n’a été recherchée : la requête a généré des combinaisons. Si nous sélectionnions les six produits, les deux lignes de campus produiraient douze combinaisons. Ce nombre est un contrôle rapide utile avant un CROSS JOIN plus grand. Un résultat beaucoup plus grand n’est correct que si chaque combinaison générée est réellement nécessaire.

Résultat / données d’exemple

name	campus
Keyboard	north
Keyboard	south
Mouse	north
Mouse	south

Comptez les combinaisons voulues avant de les générer.

Une autojointure donne deux rôles à une table

Une autojointure joint une table à elle-même avec des alias différents. C’est une jointure ordinaire, pas un mot-clé distinct. Dans categories, parent_id identifie une autre catégorie de la même table. L’alias child fournit chaque catégorie ; parent fournit celle dont elle dépend. LEFT JOIN conserve la catégorie supérieure All, bien que son parent_id soit NULL et qu’aucun parent ne puisse correspondre.

SQL / lire l’instruction une clause à la fois

```sql
SELECT child.name AS category,
       parent.name AS parent_category
FROM campus.categories AS child
LEFT JOIN campus.categories AS parent
  ON parent.category_id = child.parent_id
ORDER BY child.category_id;
```

Sens des mots-clés

Autojointure — Utiliser deux fois la même table dans une jointure, avec des alias de rôle différents.

parent_id — Identifiant de la catégorie immédiatement au-dessus de cette catégorie.

Chaque catégorie reçoit son parent immédiat

Technology et Lifestyle ont toutes deux All comme parent immédiat. Accessories dépend de Technology, tandis que Stationery dépend de Lifestyle. All reste visible avec un parent NULL, car LEFT JOIN préserve chaque ligne enfant. Le résultat a cinq lignes, car chaque catégorie peut correspondre à au plus un identifiant de parent. La même table apparaît deux fois dans la requête, mais aucune ligne enregistrée n’est copiée ou modifiée.

Résultat / données d’exemple

category	parent_category
All	NULL
Technology	All
Lifestyle	All
Accessories	Technology
Stationery	Lifestyle

WHERE filtre les lignes ; HAVING filtre les totaux

Après une jointure, l’agrégation synthétise toujours les lignes qu’elle reçoit. Une fonction d’agrégation combine les valeurs de plusieurs lignes ; SUM les additionne. GROUP BY choisit les valeurs identifiant chaque groupe. Ici, WHERE conserve d’abord les commandes payées, GROUP BY forme un groupe par commande et HAVING garde les groupes dont le chiffre d’affaires des lignes atteint 70. Les prix historiques des articles mesurent ces ventes, même si le prix actuel du produit change.

SQL / lire l’instruction une clause à la fois

```sql
SELECT o.order_id,
       SUM(i.quantity * i.unit_price) AS revenue
FROM campus.orders AS o
JOIN campus.order_items AS i USING (order_id)
WHERE o.status = 'paid'
GROUP BY o.order_id
HAVING SUM(i.quantity * i.unit_price) >= 70
ORDER BY o.order_id;
```

Sens des mots-clés

Fonction d’agrégation — Combiner plusieurs valeurs d’entrée en une valeur de synthèse.

GROUP BY — Créer des groupes de lignes ayant les mêmes valeurs de regroupement.

HAVING — Conserver ou éliminer les groupes après la formation de leurs synthèses.

Prix unitaire historique — Prix de vente stocké dans l’article de commande, pas le prix actuel du produit.

Quatre groupes de commandes payées atteignent le seuil

Les commandes 101, 102 et 107 totalisent chacune 70, tandis que la commande 104 totalise 75. Les autres commandes payées forment des groupes mais échouent à HAVING. La commande annulée 106 totaliserait 100, mais WHERE élimine ses lignes avant le regroupement : elle ne peut donc pas entrer dans ce chiffre d’affaires payé. Chaque ligne de sortie représente maintenant un total de commande, plus un article individuel.

Résultat / données d’exemple

order_id	revenue
101	70.00
102	70.00
104	75.00
107	70.00

Écrivez le filtre de lignes avant de calculer la synthèse du groupe.

À vous de jouer : chiffre d’affaires payé par étudiant

Trouvez les étudiants dont le chiffre d’affaires des lignes payées atteint au moins 100. Reliez students à orders par student_id, puis orders à order_items par order_id. Conservez les commandes payées avant le regroupement. Renvoyez full_name et la somme de quantity multiplié par le unit_price historique de l’article. Regroupez par student_id et full_name, puis appliquez le seuil avec HAVING. Triez par student_id pour faciliter la vérification.

Suivez le raisonnement

1. Joignez les trois tables avec leurs identifiants.
2. Utilisez WHERE o.status = 'paid' avant GROUP BY.
3. Regroupez par s.student_id et s.full_name.
4. Utilisez HAVING sur la même expression SUM que dans SELECT.

Choisissez l’identifiant de groupe selon la question métier, pas seulement le nom affiché.

Corrigé expliqué : trois étudiants atteignent 100

Alice, Bilal et Chloe atteignent le seuil avec des chiffres d’affaires payés de 120, 140 et 115. Diego et Eva ont chacun 50, tandis que Farah n’a aucune commande correspondante et n’entre pas dans cette jointure INNER JOIN. Les colonnes de regroupement identifient un étudiant et HAVING évalue son total terminé. Les paiements sont inutiles, car la question mesure les lignes de commande vendues, pas les transactions de paiement.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.full_name,
       SUM(i.quantity * i.unit_price) AS revenue
FROM campus.students AS s
JOIN campus.orders AS o USING (student_id)
JOIN campus.order_items AS i USING (order_id)
WHERE o.status = 'paid'
GROUP BY s.student_id, s.full_name
HAVING SUM(i.quantity * i.unit_price) >= 100
ORDER BY s.student_id;
```

Résultat / données d’exemple

full_name	revenue
Alice Martin	120.00
Bilal Amrani	140.00
Chloe Bernard	115.00

Deux tables enfants peuvent multiplier la même commande

La commande 102 comporte deux lignes d’articles et deux lignes de paiement. Chaque table a une relation un-à-plusieurs avec orders : une commande peut avoir plusieurs lignes liées. Joindre directement les articles aux paiements par order_id associe chaque article à chaque paiement. Cette multiplication s’appelle fanout. La correspondance est valide, mais elle change la granularité en paire article-paiement et rend une somme financière naïve incorrecte.

Commande 102 : lignes d’articles

product_id	quantity	unit_price	line_amount
2	2	25.00	50.00
4	1	20.00	20.00

Commande 102 : lignes de paiement

payment_id	amount
2	30.00
3	40.00

Avant d’additionner, vérifiez que chaque montant apparaît une fois.

Deux tables enfants peuvent multiplier la même commande / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

Un-à-plusieurs — Une ligne d’un côté peut correspondre à plusieurs lignes de l’autre.

Fanout — Une jointure produit plusieurs lignes de sortie pour une ligne d’origine.

Avant d’additionner, vérifiez que chaque montant apparaît une fois.

Examiner la jointure avant de se fier à son SUM

Cette requête montre délibérément les paires détaillées avant d’ajouter un agrégat. Les deux jointures correspondent sur order_id ; chaque article de la commande 102 trouve donc les deux paiements de la commande 102. Sélectionner product_id et payment_id révèle ce que représente une ligne jointe. Si nous affichions uniquement les colonnes monétaires ou utilisions immédiatement SUM, les lignes sources répétées seraient plus difficiles à remarquer.

SQL / lire l’instruction une clause à la fois

```sql
SELECT i.product_id,
       i.quantity * i.unit_price AS line_amount,
       p.payment_id, p.amount
FROM campus.order_items AS i
JOIN campus.payments AS p USING (order_id)
WHERE i.order_id = 102
ORDER BY i.product_id, p.payment_id;
```

Quatre paires transforment chaque total correct en 140

Le montant de la ligne Mouse, 50, apparaît deux fois et celui de Bottle, 20, apparaît deux fois. Leur somme vaut donc 140 au lieu du vrai total des lignes, 70. Les paiements de 30 et de 40 sont aussi répétés deux fois chacun ; les additionner produit donc le même total erroné de 140. Des totaux égaux ne prouvent pas la justesse lorsque les deux mesures ont été multipliées par la même jointure.

Résultat / données d’exemple

product_id	line_amount	payment_id	amount
2	50.00	2	30.00
2	50.00	3	40.00
4	20.00	2	30.00
4	20.00	3	40.00

Des totaux monétaires identiques peuvent partager la même erreur.

Préagréger chaque source à une ligne par commande

La préagrégation consiste à synthétiser une source avant de la joindre à une autre. Réduisez d’abord les articles à un total de lignes par commande et les paiements à un total de paiements par commande. Les deux synthèses peuvent alors correspondre une fois par commande. Une table dérivée est un SELECT entre parenthèses dans FROM ou JOIN ; son résultat sert de table temporaire pour cette requête, sans créer de table enregistrée.

Sens des mots-clés

Préagrégation — Synthétiser les lignes à la granularité nécessaire avant la jointure.

Table dérivée — SELECT entre parenthèses utilisé comme table dans une autre requête.

Sous-requête — Requête écrite dans une autre requête SQL.

Suivez le raisonnement

1. Regroupez les lignes d’articles par order_id et calculez un total de lignes.
2. Regroupez les lignes de paiement par order_id et calculez un total de paiements.
3. Joignez ces synthèses sur order_id, pour une seule correspondance par commande.

Joindre deux synthèses au niveau de la commande

Le premier SELECT entre parenthèses crée les totaux d’articles et se nomme i. Le second crée les totaux de paiements et se nomme p. Chaque GROUP BY garantit une ligne de synthèse par identifiant de commande dans sa source. Le JOIN externe compare ensuite ces identifiants. Filtrer la commande 102 hors des synthèses permet d’examiner le résultat corrigé sans changer le total calculé pour cette commande.

SQL / lire l’instruction une clause à la fois

```sql
SELECT i.order_id, i.total AS line_total,
       p.total AS payment_total
FROM (
  SELECT order_id, SUM(quantity * unit_price) AS total
  FROM campus.order_items GROUP BY order_id
) AS i
JOIN (
  SELECT order_id, SUM(amount) AS total
  FROM campus.payments GROUP BY order_id
) AS p ON p.order_id = i.order_id
WHERE i.order_id = 102;
```

La comparaison corrigée utilise 70 une fois par source

Le résultat corrigé comporte une ligne pour la commande 102 et les deux totaux indépendants valent 70. Essayez maintenant un diagnostic sans écrire de requête plus longue : la commande 101 contient deux lignes d’articles et un paiement de 70. Si ses articles et paiements sont joints directement, combien de lignes apparaissent, quelle est la somme des paiements et quelle source faut-il synthétiser pour éviter de répéter ce paiement ?

Résultat / données d’exemple

order_id	line_total	payment_total
102	70.00	70.00

Suivez le raisonnement

1. Associez chacun des deux articles de la commande 101 à son unique paiement.
2. Comptez les paires obtenues.
3. Additionnez le montant du paiement une fois par paire.

Vérifiez chaque mesure séparément ; une somme correcte ne valide pas les autres.

Corrigé expliqué : un seul paiement se répète tout de même deux fois

La commande 101 produit deux lignes jointes : chacun de ses deux articles est associé au même paiement. Additionner le montant des paiements après cette jointure donne 140, alors que seuls 70 ont été payés. Synthétiser les articles à une ligne par commande élimine la correspondance répétée. Synthétiser les deux sources à la même granularité de commande est une méthode cohérente pour comparer leurs totaux entre commandes ayant des nombres de lignes différents.

Résultat / données d’exemple

order_id	Lignes d’articles	Lignes de paiement	Lignes jointes	SUM des paiements
101	2	1	2	140.00

Alignez d’abord les sources sur la granularité de la comparaison.

Une sous-requête scalaire fournit une valeur

Une sous-requête scalaire est une requête entre parenthèses utilisée là où SQL attend une valeur. Elle doit renvoyer une colonne et au plus une ligne. Une requête sans ligne fournit NULL ; plusieurs lignes causent une erreur. MAX est un agrégat renvoyant la plus grande valeur non nulle. Nous pouvons d’abord calculer le prix maximal des produits actifs, puis utiliser cette seule valeur dans une comparaison.

SQL / lire l’instruction une clause à la fois

```sql
SELECT MAX(unit_price) AS highest_active_price
FROM campus.products
WHERE active;
```

Résultat / données d’exemple

highest_active_price
50.00

Sens des mots-clés

Scalaire — Une seule valeur plutôt qu’une table de lignes.

Sous-requête scalaire — Requête utilisée comme une valeur : une colonne et au plus une ligne.

MAX — Renvoyer la plus grande valeur non nulle du groupe.

Comparer chaque produit actif à cette valeur

Le SELECT entre parenthèses fournit le prix actif le plus élevé, 50. La requête externe examine les produits actifs et conserve ceux dont unit_price égale cette valeur. Les deux requêtes traitent des parties différentes d’une même question : l’interne calcule un seuil et l’externe renvoie les produits qui l’atteignent. Si plusieurs produits actifs étaient ex æquo au maximum, ils seraient tous renvoyés.

SQL / lire l’instruction une clause à la fois

```sql
SELECT product_id, name, unit_price
FROM campus.products
WHERE active
  AND unit_price = (
    SELECT MAX(unit_price)
    FROM campus.products
    WHERE active
  )
ORDER BY product_id;
```

Le seuil scalaire sélectionne Keyboard

Keyboard est actif et coûte 50, ce qui correspond au seuil scalaire. Les autres produits actifs coûtent moins et échouent à la comparaison d’égalité. USB Hub est inactif ; il ne peut donc pas être sélectionné, même si son prix figure dans le catalogue enregistré. Le résultat est une ligne de produit, tandis que la requête interne ne renvoyait qu’un nombre. Distinguer ces deux formes de résultat facilite la lecture des requêtes imbriquées.

Résultat / données d’exemple

product_id	name	unit_price
1	Keyboard	50.00

Une sous-requête corrélée utilise la ligne externe courante

Une sous-requête corrélée référence une colonne de la requête qui l’entoure. Ici, p nomme le produit actuellement testé et q les produits considérés pour la moyenne de sa catégorie. AVG calcule la moyenne arithmétique des valeurs non nulles. La condition q.category_id = p.category_id fait dépendre le seuil de la catégorie de p ; chaque produit est donc comparé à sa propre catégorie, plutôt qu’au catalogue entier.

SQL / lire l’instruction une clause à la fois

```sql
SELECT p.name, p.unit_price
FROM campus.products AS p
WHERE p.unit_price > (
  SELECT AVG(q.unit_price)
  FROM campus.products AS q
  WHERE q.category_id = p.category_id
)
ORDER BY p.product_id;
```

Sens des mots-clés

Sous-requête corrélée — Requête imbriquée référençant une valeur de la ligne qui l’entoure.

Requête externe — Requête environnante contenant une sous-requête.

AVG — Somme des valeurs non nulles divisée par leur nombre.

Les seuils propres aux catégories sélectionnent deux produits

Notebook coûte 10, au-dessus de la moyenne Stationery de 7.5 calculée sur Notebook et Sticker Pack. USB Hub coûte 30, au-dessus de la moyenne Accessories de 27.5 calculée sur Mouse et USB Hub. Keyboard et Bottle sont chacun seuls dans leur catégorie ; leurs prix égalent donc leurs moyennes de catégorie et échouent au test strictement supérieur. La comparaison inclut le produit courant lui-même dans la moyenne de sa catégorie.

Résultat / données d’exemple

name	unit_price
Notebook	10.00
USB Hub	30.00

Une référence corrélée change l’ensemble de comparaison pour la ligne courante.

EXISTS teste une correspondance sans la copier

EXISTS est vrai si sa sous-requête renvoie au moins une ligne et faux si elle n’en renvoie aucune. NOT EXISTS inverse ce test. Utilisez-les si la question demande si une relation est présente ou absente, plutôt que d’afficher chaque ligne correspondante. Le SELECT 1 interne est une convention : EXISTS teste la présence de lignes ; la valeur sélectionnée elle-même ne compte donc pas ici.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.student_id, s.full_name
FROM campus.students AS s
WHERE EXISTS (
  SELECT 1
  FROM campus.orders AS o
  WHERE o.student_id = s.student_id
    AND o.status = 'paid'
)
ORDER BY s.student_id;
```

Sens des mots-clés

EXISTS — Vrai lorsque la requête imbriquée produit au moins une ligne.

NOT EXISTS — Vrai lorsque la requête imbriquée ne produit aucune ligne.

SELECT 1 dans EXISTS — Constante conventionnelle, car seule la présence de lignes est testée.

Chaque étudiant admissible apparaît une fois

Cinq étudiants ont au moins une commande payée. Alice, Bilal et Chloe en ont chacun plusieurs, mais leurs lignes d’étudiants sont renvoyées une fois, car EXISTS teste seulement une correspondance. Farah n’a aucune commande ; le test est donc faux pour elle. La granularité reste un étudiant. Aucun DISTINCT n’est nécessaire pour corriger la répétition des commandes, car la requête ne crée jamais de lignes de sortie au niveau des commandes.

Résultat / données d’exemple

student_id	full_name
1	Alice Martin
2	Bilal Amrani
3	Chloe Bernard
4	Diego Costa
5	Eva Dupont

Choisissez EXISTS lorsque l’information nécessaire est l’existence d’une correspondance.

À vous de jouer : étudiants sans commande payée

Écrivez le rapport complémentaire : listez les étudiants sans commande payée. Gardez la même requête externe students et les mêmes conditions corrélées sur les commandes, mais remplacez le test de présence par NOT EXISTS. Renvoyez student_id et full_name, triés par student_id. Expliquez pourquoi un étudiant n’ayant que des commandes annulées doit être retenu, tandis qu’un étudiant ayant des commandes annulées et payées ne doit pas l’être.

Suivez le raisonnement

1. Partez d’une ligne externe pour chaque étudiant.
2. Dans la sous-requête, exigez un student_id correspondant et le statut paid.
3. Utilisez NOT EXISTS pour conserver les étudiants pour lesquels cette requête interne ne trouve rien.

Corrigé expliqué : l’absence signifie aucune correspondance payée

La requête interne cherche uniquement les commandes payées appartenant à l’étudiant courant. NOT EXISTS conserve l’étudiant seulement si cette recherche ne trouve aucune ligne. Farah est retenue dans le jeu de données de référence. Eva ne l’est pas : sa commande annulée n’annule pas sa commande payée 109. Voilà pourquoi chercher une commande non payée exprimerait une condition différente et pourrait inclure Eva à tort.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.student_id, s.full_name
FROM campus.students AS s
WHERE NOT EXISTS (
  SELECT 1
  FROM campus.orders AS o
  WHERE o.student_id = s.student_id
    AND o.status = 'paid'
)
ORDER BY s.student_id;
```

Résultat / données d’exemple

student_id	full_name
6	Farah Idrissi

IN accepte une requête à une colonne, mais NULL compte

IN peut comparer une valeur aux lignes renvoyées par une sous-requête à une colonne, comme au jour 1 avec une liste littérale. NOT IN demande que la valeur diffère de chaque valeur renvoyée. Une comparaison avec NULL vaut inconnu ; un NULL dans les résultats de recherche peut donc empêcher NOT IN de devenir vrai. Ces petits résultats VALUES contiennent délibérément NULL pour révéler ce comportement.

SQL / lire l’instruction une clause à la fois

```sql
SELECT v.n,
       v.n IN (VALUES (1), (NULL)) AS in_set,
       v.n NOT IN (VALUES (1), (NULL)) AS not_in_set
FROM (VALUES (1), (6)) AS v(n)
ORDER BY v.n;
```

Sens des mots-clés

IN (subquery) — Tester l’égalité avec les valeurs renvoyées par une requête à une colonne.

NOT IN — Tester qu’aucune valeur renvoyée n’est égale, NULL influant sur la réponse.

Inconnu — Résultat NULL d’une comparaison dont on ne peut déterminer la vérité.

Inconnu n’est pas une condition acceptée par WHERE

Le candidat 1 a une valeur connue égale ; IN est donc vrai et NOT IN faux. Le candidat 6 n’a aucune égalité connue, mais NULL pourrait représenter 6 ou une autre valeur ; les deux tests sont donc inconnus. En SQL, NOT inconnu reste inconnu. WHERE conserve seulement les conditions vraies ; un test WHERE n NOT IN écarterait donc ici le candidat 6, même si 6 ne figure pas parmi les valeurs connues.

Résultat / données d’exemple

n	in_set	not_in_set
1	TRUE	FALSE
6	NULL	NULL

WHERE écarte une condition inconnue comme une condition fausse.

NOT EXISTS précise l’égalité absente

Pour une question d’absence, NOT EXISTS peut exprimer directement la règle de correspondance. La sous-requête ci-dessous cherche une ligne dont n égale la valeur candidate. Une comparaison avec NULL ne compte pas comme correspondance, car WHERE exige vrai. Le candidat 6 est donc conservé. C’est la règle voulue ici : les valeurs bloquées inconnues ne bloquent pas un candidat connu. Décidez explicitement de cette règle si les valeurs manquantes ont un sens métier.

SQL / lire l’instruction une clause à la fois

```sql
SELECT v.n
FROM (VALUES (1), (6)) AS v(n)
WHERE NOT EXISTS (
  SELECT 1
  FROM (VALUES (1), (NULL)) AS b(n)
  WHERE b.n = v.n
)
ORDER BY v.n;
```

Résultat / données d’exemple

n
6

ANY exige un succès ; ALL exige tous les succès

Un opérateur de comparaison tel que > suivi de ANY compare une valeur à chaque valeur renvoyée et réussit si au moins une comparaison est vraie. ALL exige que toutes les comparaisons soient vraies. Ici, > ANY signifie supérieur à au moins un seuil, tandis que > ALL signifie supérieur aux deux seuils. La sous-requête doit renvoyer une colonne. SOME est une autre écriture de ANY ; IN est la forme d’égalité, = ANY.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price,
       unit_price > ANY (VALUES (20), (30)) AS any_gt,
       unit_price > ALL (VALUES (20), (30)) AS all_gt
FROM campus.products
ORDER BY product_id;
```

Sens des mots-clés

ANY — Au moins une comparaison doit être vraie.

ALL — Chaque comparaison doit être vraie ; un ensemble de comparaison vide est aussi accepté.

SOME — Synonyme de ANY.

Opérateur — Symbole tel que > ou = qui effectue ici une comparaison.

Un prix de 30 satisfait ANY mais pas ALL

Keyboard à 50 dépasse 20 et 30 ; les deux tests sont donc vrais. Mouse à 25 et USB Hub à 30 dépassent 20, ce qui suffit pour ANY, mais aucun ne dépasse 30 ; ALL est donc faux. Bottle égale 20 et ne dépasse aucun seuil. Strictement supérieur n’inclut pas l’égalité. Remplacer > par >= rendrait vrai le résultat ALL de USB Hub.

Résultat / données d’exemple

name	unit_price	any_gt	all_gt
Keyboard	50.00	TRUE	TRUE
Mouse	25.00	TRUE	FALSE
Notebook	10.00	FALSE	FALSE
Bottle	20.00	FALSE	FALSE
USB Hub	30.00	TRUE	FALSE
Sticker Pack	5.00	FALSE	FALSE

Traduisez ANY et ALL en comparaisons séparées avant de combiner leurs valeurs de vérité.

UNION empile les résultats ; ALL conserve les lignes répétées

Une opération ensembliste combine verticalement des résultats de requêtes complets, au lieu de rattacher des colonnes comme une jointure. Les deux requêtes doivent renvoyer le même nombre de colonnes, avec des types compatibles à chaque position. UNION élimine les lignes entières répétées ; UNION ALL les conserve. Ces deux petites requêtes renvoient des nombres pour rendre la différence visible. Le ORDER BY final trie la sortie combinée, pas seulement la seconde entrée.

SQL / lire l’instruction une clause à la fois

```sql
SELECT n FROM (VALUES (1), (1), (2)) AS a(n)
UNION
SELECT n FROM (VALUES (2), (3)) AS b(n)
ORDER BY n;
SELECT n FROM (VALUES (1), (1), (2)) AS a(n)
UNION ALL
SELECT n FROM (VALUES (2), (3)) AS b(n)
ORDER BY n;
```

Sens des mots-clés

Opération ensembliste — Combiner des lignes entières de deux résultats de requêtes compatibles.

UNION — Combiner les résultats et éliminer les lignes entières répétées.

UNION ALL — Combiner les résultats en conservant chaque occurrence d’une ligne.

Types compatibles — Types que PostgreSQL peut résoudre en un type de sortie commun.

Trois lignes distinctes ou cinq occurrences enregistrées

UNION renvoie 1, 2 et 3 une fois chacun. UNION ALL renvoie cinq lignes, car les deux occurrences de 1 et celles de 2 sont conservées. Aucun choix n’est universellement correct. Une liste de contacts uniques peut nécessiter UNION, tandis que combiner des transactions exige généralement de préserver leurs occurrences. L’élimination des doublons compare toute la ligne sélectionnée ; ajouter une colonne peut donc changer les lignes considérées comme égales.

UNION : valeurs distinctes

n
1
2
3

Déterminez si les occurrences des lignes ont un sens avant de les éliminer.

Trois lignes distinctes ou cinq occurrences enregistrées / résultat commenté

UNION renvoie 1, 2 et 3 une fois chacun. UNION ALL renvoie cinq lignes, car les deux occurrences de 1 et celles de 2 sont conservées. Aucun choix n’est universellement correct. Une liste de contacts uniques peut nécessiter UNION, tandis que combiner des transactions exige généralement de préserver leurs occurrences. L’élimination des doublons compare toute la ligne sélectionnée ; ajouter une colonne peut donc changer les lignes considérées comme égales.

UNION ALL : toutes les occurrences

n
1
1
2
2

Déterminez si les occurrences des lignes ont un sens avant de les éliminer.

Trois lignes distinctes ou cinq occurrences enregistrées / résultat commenté

UNION renvoie 1, 2 et 3 une fois chacun. UNION ALL renvoie cinq lignes, car les deux occurrences de 1 et celles de 2 sont conservées. Aucun choix n’est universellement correct. Une liste de contacts uniques peut nécessiter UNION, tandis que combiner des transactions exige généralement de préserver leurs occurrences. L’élimination des doublons compare toute la ligne sélectionnée ; ajouter une colonne peut donc changer les lignes considérées comme égales.

UNION ALL : toutes les occurrences

n
3

Déterminez si les occurrences des lignes ont un sens avant de les éliminer.

INTERSECT conserve les lignes communes ; EXCEPT soustrait

INTERSECT renvoie les lignes présentes dans les deux résultats. EXCEPT renvoie celles présentes dans le premier mais absentes du second ; son sens compte donc. Leurs formes par défaut éliminent les lignes entières répétées. Les entrées sont ici 1, 1, 2 à gauche et 2, 3 à droite. Exécutez les deux instructions complètes séparément et prévoyez leurs sorties avant de regarder la réponse.

SQL / lire l’instruction une clause à la fois

```sql
SELECT n FROM (VALUES (1), (1), (2)) AS a(n)
INTERSECT
SELECT n FROM (VALUES (2), (3)) AS b(n)
ORDER BY n;
SELECT n FROM (VALUES (1), (1), (2)) AS a(n)
EXCEPT
SELECT n FROM (VALUES (2), (3)) AS b(n)
ORDER BY n;
```

Sens des mots-clés

INTERSECT — Conserver les lignes entières présentes dans les deux entrées.

EXCEPT — Conserver les lignes entières de la première entrée absentes de la seconde.

À vous de jouer : prévoir le sens de EXCEPT

Utilisez les mêmes entrées : A contient 1, 1, 2 et B contient 2, 3. Écrivez la sortie de A INTERSECT B et A EXCEPT B. Inversez ensuite la soustraction et prévoyez B EXCEPT A. Enfin, expliquez pourquoi le 1 répété ne produit pas deux lignes dans le résultat EXCEPT par défaut. Vérifiez votre raisonnement en échangeant les deux instructions SELECT complètes.

Suivez le raisonnement

1. Identifiez les valeurs présentes des deux côtés.
2. Retirez de A les valeurs de B pour A EXCEPT B.
3. Inversez les deux sources et répétez la soustraction.

Corrigé expliqué : la première entrée fixe le sens

La valeur commune est 2 ; INTERSECT renvoie donc une ligne contenant 2. A EXCEPT B renvoie une ligne contenant 1, tandis que B EXCEPT A renvoie une ligne contenant 3. EXCEPT élimine par défaut les lignes de résultat répétées ; les deux occurrences de 1 dans A ne produisent donc qu’une ligne de sortie. Inverser les entrées change la soustraction, mais pas les valeurs communes.

A INTERSECT B

n
2

A EXCEPT B

n
1

Lisez EXCEPT comme premier résultat moins second résultat.

Corrigé expliqué : la première entrée fixe le sens / résultat commenté

La valeur commune est 2 ; INTERSECT renvoie donc une ligne contenant 2. A EXCEPT B renvoie une ligne contenant 1, tandis que B EXCEPT A renvoie une ligne contenant 3. EXCEPT élimine par défaut les lignes de résultat répétées ; les deux occurrences de 1 dans A ne produisent donc qu’une ligne de sortie. Inverser les entrées change la soustraction, mais pas les valeurs communes.

B EXCEPT A

n
3

Lisez EXCEPT comme premier résultat moins second résultat.

« A acheté chaque produit requis » est une question portant sur tous les éléments

La division relationnelle désigne la recherche de candidats liés à chaque membre d’un ensemble requis. C’est un modèle de question, pas un mot-clé DIVIDE. Notre ensemble requis ne contient que deux produits : Keyboard et Mouse. Un étudiant est retenu si ses commandes payées contiennent les deux produits, éventuellement dans des commandes différentes. Acheter d’autres produits est autorisé. Nous exprimerons la règle ainsi : « aucun produit requis ne manque pour cet étudiant ».

Résultat / données d’exemple

product_id requis	product
1	Keyboard
2	Mouse

Sens des mots-clés

Division relationnelle — Trouver les candidats satisfaisant une relation pour chaque membre d’un ensemble requis.

Ensemble requis — Exigences explicitement énumérées que chaque candidat retenu doit couvrir.

Traduisez « chaque exigence » en « aucune exigence ne manque ».

Deux tests d’absence vérifient le petit ensemble requis

Pour chaque étudiant, le NOT EXISTS externe cherche un produit requis pour lequel la recherche interne d’achat échoue. Le NOT EXISTS interne est vrai si cet étudiant n’a aucune ligne de commande payée pour le produit requis courant. Si une seule exigence manque, l’étudiant est écarté. Tous les mots-clés ont déjà été présentés : l’étape supplémentaire consiste à suivre simultanément l’alias étudiant s et l’alias de produit requis r.

SQL / lire l’instruction une clause à la fois

```sql
SELECT s.student_id, s.full_name
FROM campus.students AS s
WHERE NOT EXISTS (
  SELECT 1 FROM (VALUES (1), (2)) AS r(product_id)
  WHERE NOT EXISTS (
    SELECT 1
    FROM campus.orders AS o
    JOIN campus.order_items AS i USING (order_id)
    WHERE o.student_id = s.student_id
      AND o.status = 'paid'
      AND i.product_id = r.product_id))
ORDER BY s.student_id;
```

Suivez le raisonnement

1. Choisissez un étudiant de la requête externe.

2. Pour chaque produit requis, cherchez dans les lignes de commande payées de cet étudiant.

3. Écartez l’étudiant si un produit requis n’a aucune ligne correspondante.

Chloe couvre les deux exigences ; vérifiez la granularité

Chloe est la seule étudiante retenue : la commande payée 104 contient Keyboard et Mouse. Alice et Bilal manquent chacun une exigence ; Diego, Eva et Farah ne couvrent pas non plus les deux. Un étudiant correspondant est renvoyé une fois, car la source externe est students et les tests EXISTS ne la multiplient pas. Si l’ensemble requis était vide, chaque étudiant serait retenu, puisqu’aucune exigence manquante ne permettrait de l’écarter.

Résultat / données d’exemple

student_id	full_name
3	Chloe Bernard

Énoncez l’ensemble requis, la règle de correspondance et la granularité de sortie avant de composer du SQL imbriqué.